

Typen

Tim Schmitz
21.02.2024



Inhalt

Konzeptionelle Hierarchie	3
Elementare Objekttypen	3
Zahlen	3
Strings	4

Objekttypen

Der Objektbegriff ist universell im Sprachkonzept von Python. Objekte sind entweder Built-In-Konstrukte der Sprache oder eigendefiniert. Zusammen mit dem Objektbegriff sind die dazugehörigen Operationen zu sehen.

Konzeptionelle Hierarchie

Python-Programme können gemäß folgender Reihenfolge in immer einfachere Bestandteile dekomponiert werden:

1. Programme bestehen aus Modulen
2. Module enthalten Statements oder Programm-Anweisungen
3. Anweisungen enthalten Ausdrücke
4. Ausdrücke erzeugen und verarbeiten Objekte

Dabei sind Built-In-Konstrukte eine strukturelle Hilfe, da sie vielfach Routinearbeiten übernehmen wie die Auslagerung und Allokation von Speichern, die Implementierung von Such- und Zugriffsalgorithmen usw. So haben sie weitere Vorteile wie

- Einfachere Codierung für viele Problem domänen wie Collections (Listen) oder Such-Tabellen (Dictionaries)
- Selbstentwickelte Erweiterungen basieren oft auf eingebauten Datentypen
- Effizientere Lösungen statt selbst programmierter Varianten
- Standardisierung

Elementare Objekttypen

Die folgende Tabelle umfasst die elementaren Objekttypen in Python. Darunter versteht man nicht nur die elementaren Datentypen, sondern auch komplexere, eingebaute Objekttypen wie Listen oder Dictionaries sowie höhere Abstraktionsstufen.

Objekttyp	Beispiele für Repräsentanten
Zahl	1234, 3,1415m 3+4j, 0b111, Decimal()
String	'Spam', "Bob's", b'a\x01c'
Listen	[1,2,3,4,5], list(range(10))
Dictionaries	{'#1' : 'spam', '#2' : 'yum'}, dict(hours=10)
Dateien	open('file.txt'), open(r'C:\text.bin','wb')
Mengen	set('abc'), {'a','b','c'}
Sonstige	Boolean, None
Programm-Units	Funktionen, Module, Klassen

Tabelle 1: Elementare Objekttypen in Python

Neben den einfachen Objekttypen sind auch Programm-Einheiten wie Funktionen, Module und Klassen verzeichnet. Diese Einheiten werden mit Anweisungen und Ausdrücken wie `def`, `class`, `import` und `lambda` erzeugt. Außerdem bietet Python auch noch implementierte Objekte wie kompilierten Code an.

Objekte wie etwa Strings sind durch Anführungsstriche gekennzeichnet, Listen durch eckige und Dictionaries durch geschweifte Klammern. An Objekte sind Operationen gebunden, wie String-Operationen an Strings und List-Operationen an Listen. Daher ist Python durch diese automatische dynamisch typisiert, aber auch stark typisiert, denn eine Operation kann nur auf dem zugrundeliegenden Objekttyp operieren.

Zahlen

Zahlen sind elementare Basistypen. Es gibt folgende numerische Ausprägungen:

- Integers sind ganzzahlige Datentypen
- Gleitkomma-Zahlen haben einen Bruchteil
- Komplexe Zahlen haben einen imaginären Anteil
- Dezimalzahlen haben einen festen Dezimalpunkt

- Rationale Zahlen bestehen aus Zählern und Nennern

Die folgenden Operationen sind elementare Operationen auf elementaren numerischen Datentypen.

```
#!/usr/bin/python3
# numbers.py

# Ganzzahlige Addition
summand1 = 123
summand2 = 222
summe = summand1 + summand2
print(summe)

# Gleitkomma-Multiplikation
multiplikator = 1.5
multiplikand = 4
produkt = multiplikator * multiplikand
print(produkt)

# Exponentiation
basis = 2
exponent = 100
potenz = basis ** exponent
print(potenz)
```

Die Länge einer Zahl wird über die Längenfunktion berechnet, dafür wird die Zahl über die Built-In-Funktion str auf einen String-Datentyp gecastet.

```
# Laenge einer Zahl durch casting
hochzahl = 2 ** 1000000
laenge = len(str(hochzahl))
print(laenge)
```

Mathematische Module helfen bei der Berechnung komplexerer Ausdrücke und liefern nützliche Werkzeuge wie in den folgenden Beispielen, in denen fortgeschrittene mathematische Funktionen und der Gebrauch der Zufallsfunktion behandelt werden:

```
#!/usr/bin/python2
# math.py

import math
import random

# Berechnung der Kreisszahl Pi
print(math.pi)

# Berechnung der Wurzel aus 85
print(math.sqrt(85))

# Zufallszahl zwischen 0 und 1
print(random.random())

# Zufallszahl aus einer Menge vorgegebener Zahlen
print (random.choice([1,2,3,4]))
```

Strings

Strings sind Sequenzen von Zeichen (Characters). Man kann zahlreiche einfache Operationen auf Strings vornehmen.

Einfache String-Operationen

Das folgende Beispiel zeigt simple Operationen auf einem einfachen String.

```
#!/usr/bin/python3
# strings.py

# Deklaration
string = 'Spam'

# Länge, erstes und zweites Zeichen
print(len(string))      # 4
print(string[0])        # S
print(string[1])        # p

# Letzes und vorletzes Zeichen
print(string[-1])       # m
print(string[-2])       # a

# Offsets
print(string[1:3])      # pa
print(string[1:])       # pam
print(string[:3])       # Spa
print(string[:])        # Spam
```

```
# Konkatenation und Repetition
print(string+'xyz')    # Spamxyz
print(string*2)        # SpamSpam
```

String-Manipulationen

Strings sind wie andere Basistypen unveränderbar. Aber sie können über andere Konstrukte (Listen, Byte-Arrays) manipuliert werden wie das folgende Beispiel zeigt.

```
#!/usr/bin/python3
# stringsManipulation.py

# String-Casting zu Liste
string = 'shruberry'
stringList = list(string)
print(stringList)    # ['s', 'h', 'r', 'u', 'b', 'e', 'r', 'r', 'y']

# Manipulation und 'recasting'
stringList[1] = 'c'
stringList = ''.join(stringList)
print(stringList)    # scruberry

# Manipulation über Bytelisten
byteList = bytearray(b'spam')    # Hybrid
byteList.extend(b'eggs')
print(byteList)    # bytearray(b'spameggs')
print(byteList.decode())    # spameggs
```

Suchen und Ersetzen

String-spezifische Methoden sind ebenfalls leicht anwendbar. Die Methode find ist eine Suchoperation auf eine Substring und die Methode replace ersetzt Strings auf Basis einer globalen Suche.

```
#!/usr/bin/python3
# stringsMethods.py

# Methode find sucht auf einen Substring
string = 'Spam'
print(string.find('pa'))    # 1

# Die Methode replace ersetzt einen Suchstring durch einen Ersatz
print(string.replace('pa','XYZ'))    # SXYZm

# Uppercase und Alphanumeric-Test
print(string.upper())    # SPAM
print(string.isalpha())    # True
```

Formatierungen

Weitere Methoden, die von den String- und Number-Objekttypen unterstützt werden, sind Formatierungsmethoden

```
#!/usr/bin/python3
# stringsNumbsFormat.py

# Formatierung von Strings
string = '%s, eggs, and %s'
stringFormat = string % ('spam','SPAM!')
print(stringFormat)

# Generische Formatierung mit %
# Ersetze die Platzhalter
# spam, eggs, and SPAM!

# Formatierung von Zahlen (in 1000er Schritten)
floatingNumber = 296999.2567
floatingNumberFormat = '{:,.2f}'.format(floatingNumber)
print(floatingNumberFormat)

# Tausender Kennzeichen und
# Abrunden zweite Stelle hinter Punkt
# 296,999.26

# Abrunden
piNumber = 3.14159
roundPiNumber = '%.2f' % piNumber
print(roundPiNumber)

# Abrunden zweite Stelle hinter Punkt
# 3.14

# Auffüllen
negativeNumber = -42
paddingNegativeNumber = '%+05d' % negativeNumber
print(paddingNegativeNumber)

# Auffüllen mit Nullen bis zur 5.Stelle
# -0042
```